

Rapport de Sécurité

Audit de l'Accès aux Commandes de Pompage

Projet Urba-Drain Agadir

Amira Bahanni · Hicham Baba

Filière SITCN · ENSIASD Taroudant

Avril 2026

Ce rapport traite la tâche de sécurité :

Audit de l'accès aux commandes de pompage : éviter qu'un pirate puisse bloquer l'évacuation.

Deux versions : **Version Sans IA** (Flask + PyMySQL) et **Version Avec IA** (Flask + SQLAlchemy + JWT).

Table des matières

1	Contexte	2
2	Problème identifié dans la Version Sans IA	2
2.1	Description	2
2.2	Conséquence concrète	3
2.3	Correction appliquée	3
3	Problème identifié dans la Version Avec IA	4
3.1	Description	4
3.2	Correction appliquée	4
4	Tableau récapitulatif	5
5	Réponse à la question de défense	5
6	Conclusion	5

Contexte

Le projet Urba-Drain Agadir surveille le réseau pluvial de la ville. Des capteurs mesurent les niveaux d'eau dans les quartiers, et des pompes assurent l'évacuation lorsque les niveaux deviennent dangereux.

Le cahier des charges impose une tâche de sécurité précise : s'assurer qu'un attaquant ne puisse pas bloquer l'évacuation en prenant le contrôle des pompes.

Ce rapport décrit le problème trouvé dans chaque version et la correction appliquée.

Problème identifié dans la Version Sans IA

Description

Fichier concerné : `team_without_ia/backend/app/routes/pompes.py`

En lisant le fichier, on constate que les deux routes qui contrôlent les pompes ne vérifient pas l'identité de la personne qui appelle :

Code extrait du fichier — routes non protégées :

```
@bp.route('/<int:pompe_id>/activer', methods=['POST'])
def activer_pompe(pompe_id):
    # Aucun controle d'accès -- tout le monde peut
    # appeler cette route
    data = request.get_json() or {}
    utilisateur = data.get('utilisateur') # vient du
    # body, falsifiable
    success = pompe_service.activer_pompe(pompe_id,
    utilisateur)
    ...

@bp.route('/<int:pompe_id>/desactiver', methods=['POST',
])
def desactiver_pompe(pompe_id):
    # Meme probleme -- aucune authentication
    data = request.get_json() or {}
    utilisateur = data.get('utilisateur')
    success = pompe_service.desactiver_pompe(pompe_id,
    utilisateur)
    ...
```

N'importe qui sur le réseau peut envoyer une requête HTTP à ces adresses et désactiver les pompes, sans avoir de compte, sans mot de passe. L'identité de l'utilisateur vient du corps de la requête, donc on peut écrire n'importe quel nom dedans.

Conséquence concrète

Un attaquant peut désactiver toutes les pompes en quelques secondes avec un simple script. Pendant un épisode pluvieux, cela peut provoquer des inondations dans les zones à risque élevé comme Anza ou Hay Mohammadi.

Correction appliquée

On a ajouté une vérification de clé API sur les deux routes. Sans cette clé dans le header de la requête, le serveur refuse immédiatement avec une erreur 401.

Code corrigé :

```
import os

def check_api_key():
    """Verifie la cle API dans le header de la requete"""
    key = request.headers.get('X-API-Key', '')
    expected = os.getenv('SECRET_KEY', 'dev-secret-key-urba-drain-2026')
    return key == expected

@bp.route('/<int:pompe_id>/activer', methods=['POST'])
def activer_pompe(pompe_id):
    if not check_api_key():
        return jsonify({'error': 'Acces non autorise'}), 401
    data = request.get_json() or {}
    utilisateur = data.get('utilisateur')
    success = pompe_service.activer_pompe(pompe_id, utilisateur)
    ...

@bp.route('/<int:pompe_id>/desactiver', methods=['POST'])
def desactiver_pompe(pompe_id):
    if not check_api_key():
        return jsonify({'error': 'Acces non autorise'}), 401
    data = request.get_json() or {}
    utilisateur = data.get('utilisateur')
    success = pompe_service.desactiver_pompe(pompe_id, utilisateur)
    ...
```

Maintenant, une requête sans la clé correcte reçoit une erreur 401 avant d'atteindre la logique de la pompe.

Problème identifié dans la Version Avec IA

Description

Fichier concerné : `team_of_ia/backend/app/routes/pompes.py`

La version avec IA utilise JWT, ce qui est bien. Mais on a remarqué que le niveau de rôle minimum pour contrôler les pompes est **TECHNICIEN** :

Code extrait du fichier :

```
@pompes_bp.route("/<int:pompe_id>/toggle", methods=["PUT",
    ""])
@jwt_required()
@require_min_role("TECHNICIEN")    # niveau trop bas
def toggle_pompe(pompe_id):
    ...
```

Le rôle **TECHNICIEN** dans ce système correspond aux agents de maintenance qui interviennent physiquement sur les équipements. Leur travail est de réparer les pompes, pas de les activer ou désactiver en opération. Les permettre de contrôler les pompes via l'API crée un risque : si le compte d'un technicien est compromis, l'attaquant peut contrôler le pompage.

Correction appliquée

On a modifié une seule ligne pour exiger le rôle **OPERATEUR** minimum, ce qui est le bon niveau pour le contrôle opérationnel des pompes :

Code corrigé :

```
@pompes_bp.route("/<int:pompe_id>/toggle", methods=["PUT",
    ""])
@jwt_required()
@require_min_role("OPERATEUR")    # correction :
    OPERATEUR minimum
def toggle_pompe(pompe_id):
    ...
```

Désormais, seuls les comptes avec le rôle **OPERATEUR** ou **ADMIN** peuvent activer ou désactiver une pompe. Un compte **TECHNICIEN** ou **LECTEUR** reçoit une erreur 403.

Tableau récapitulatif

Version	Problème	Correction	Statut
Sans IA	Routes <code>/activer</code> et <code>/desactiver</code> sans authentification	Ajout d'une vérification de clé API	Corrigé
Avec IA	Rôle <code>TECHNICIEN</code> peut contrôler les pompes	Rôle minimum changé à <code>OPERATEUR</code>	Corrigé

Réponse à la question de défense

Question : *Comment évitez-vous qu'un pirate puisse bloquer l'évacuation en prenant le contrôle des pompes ?*

Version sans IA : Les routes de contrôle des pompes étaient ouvertes à tout le monde. On a ajouté une vérification de clé API. Sans cette clé dans le header, la requête est rejetée immédiatement avec une erreur 401, avant même d'exécuter le moindre code.

Version avec IA : L'authentification JWT était déjà en place. On a corrigé le niveau de rôle minimum pour le contrôle des pompes : `TECHNICIEN` → `OPERATEUR`. Un technicien gère la maintenance physique, pas le contrôle opérationnel. Cette séparation réduit la surface d'attaque : même si le compte d'un technicien est compromis, l'attaquant ne peut pas toucher aux pompes.

Conclusion

Les deux modifications apportées répondent directement à la tâche de sécurité imposée par le cahier des charges. Dans la version sans IA, les routes de pompage étaient sans protection — n'importe qui pouvait les appeler. Dans la version avec IA, la protection JWT était bien là mais le niveau de rôle n'était pas adapté à la logique du système.

Les deux corrections sont simples, visibles dans le code, et directement explicables.

Amira Bahanni · Hicham Baba · Filière SITCN · ENSIASD Taroudant · 2025-2026